CSC3066
Deep Learning

Training Deep Neural Network: Gradient Descent & Backpropagation

Anna Jurek-Loughrey

What's to come today

- Gradient Descent
- Backpropagation

Neural Network

Gradient Descent

Neural Network: Learning

- Training of MLP is much more complicated.
- With a simple perceptron, we could easily evaluate how to change the weights according to the error.
- Now there are many different connections, each in a different layer of the network.
- How does one know how much each neuron or connection contributed to the overall error of the network?

Neural Network: Learning

- Each example x from the training set is accompanied by a label y.
- The training examples specify directly what the output layer must do at each point x – it must produce a value that is close to y.
- The behaviour of the other layers is not directly specified by the training data.
- The learning algorithm must decide how to use those layers to produce the desire output, i.e. what values should be assigned to their weights (parameters)

Training Neural Network: Optimisation Problem

- Optimisation refers to the task of either minimising or maximising some function $f(W)$ by alerting the value of W.
- In other words, during the optimisation process we are looking for a value W^*such that:

$$W^* = \mathbf{argmin}\, f(W) \text{ (minimisation)}$$
$$W^* = \mathbf{argmax}\, f(W) \text{ (maximisation)}$$

- The function we want to minimise or maximise is referred to as objective function.
- When the function is minimised it can also be called cost function, loss function, or error function → we will focus on minimisation problem.

Gradient Optimisation Method

- The derivative is useful for minimising functions because it tells us how to change W in order to make a small improvement in $f(W)$.
- We know that $f(W - \epsilon sign(f'(W)))$ is less than $f(W)$.
- We can thus reduce $f(W)$ by moving W in small steps with the opposite sign of the derivative.
- This technique is called gradient descent.

Gradient Optimisation Method

- In the context of deep learning, we often optimise functions that may have many local minima that are not optimal and many saddle points surrounded by very flat region.
- All of this makes the optimisation very challenging, especially when the input to the function is multidimensional.
- Therefore, while training deep learning models, we usually settle for finding a value of f that is very low but not necessarily minimal.

Gradient Optimisation Method: Multiple inputs

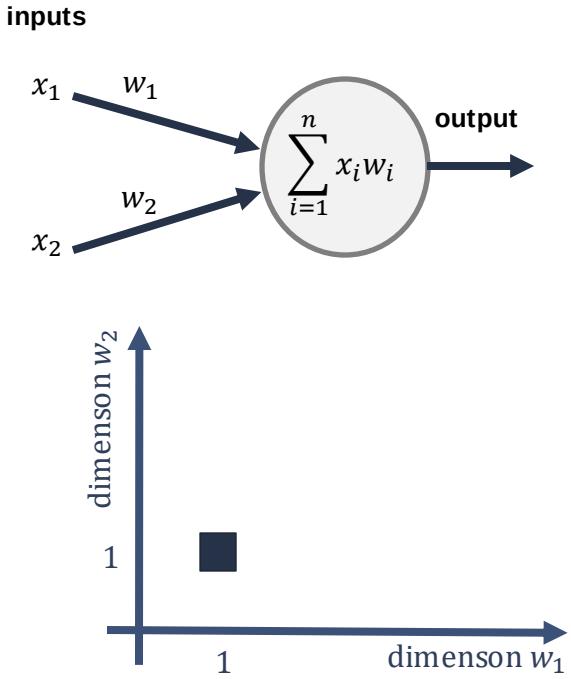
- We often minimise function that have multiple inputs: $f: R^n \rightarrow R$
- In order to minimise function with multiple inputs we use the concept of partial derivatives.
- The partial derivative $\frac{\partial}{\partial W_i} f(W)$ measures how f changes as only the variable W_i increases at point $\mathbf{W}$.

Gradient Optimisation Method: Multiple inputs

- The vector containing all partial derivatives is called gradient of f and is denoted as $\nabla_W f(W)$
- Element i of the gradient is the partial derivative of f with respect to W_i.
- In multiple dimensions, critical points are points where every element of the gradient is equal to zero.

Gradient Optimisation Method: Multiple inputs

- Let's consider a very simple network with just 2 inputs, and one neuron.
- The two weights w_1 and w_2 have particular values, let's say (1,1).
- We can visualise the weights as a point in a 2-dimensional plane:



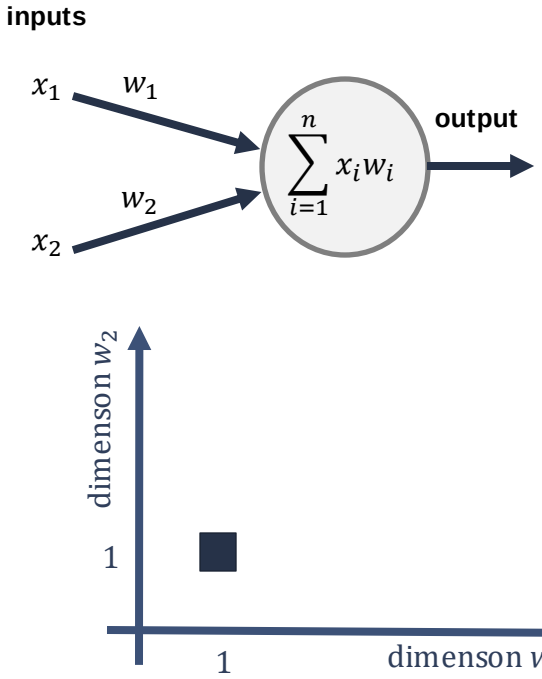
Gradient Optimisation Method: Multiple inputs

- Now, suppose we present a particular item to the network and calculate the error

- Suppose the error is defined as half the square of the difference between the observed and expected output:

$$E(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

This is a standard choice; we will see why.

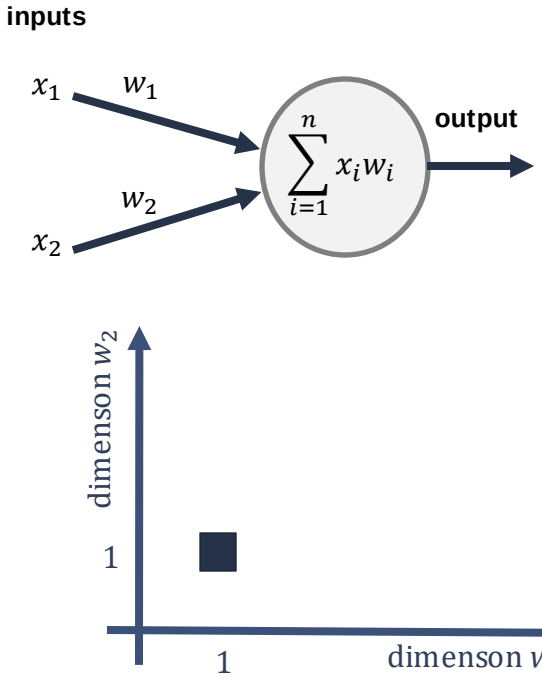


Gradient Optimisation Method: Multiple inputs

- Suppose for our particular item, the output was 0.4 and the correct output is 1.0; then:

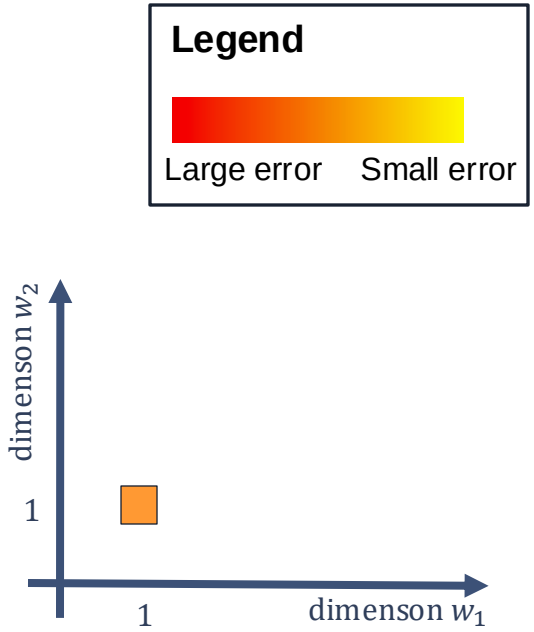
$$E = \frac{1}{2}(1 - 0.4)^2 = 0.18$$

- This is the particular error value for the current training item, when the weights are $w_1 = 1$ and $w_2 = 1$.



Gradient Optimisation Method: Multiple inputs

- Let's colour-code the point in weight-space by what the Error is for these weight values



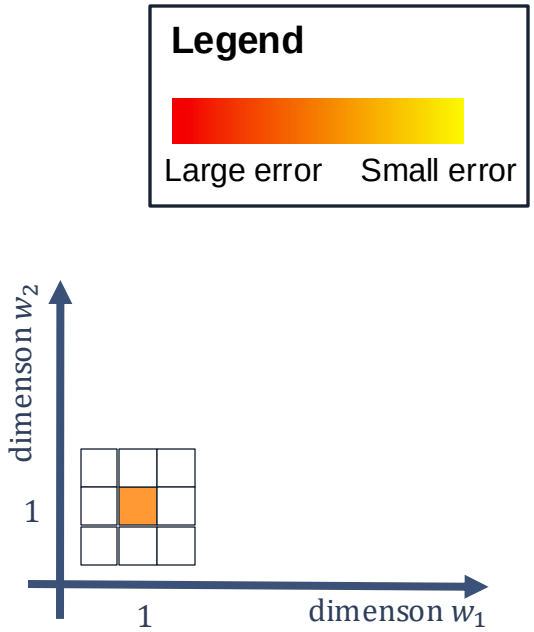
Gradient Optimisation Method: Multiple inputs

- Let's next consider points in the feature space that are close to our original point (1,1). E.g.

$$w_1 \in \{0.9, 1.0, 1.1\}$$

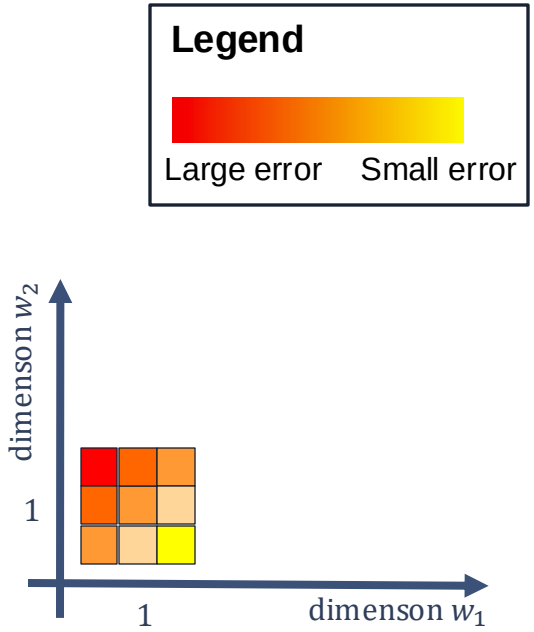
$$w_2 \in \{0.9, 1.0, 1.1\}$$

- We are considering 9 weight combinations in total.



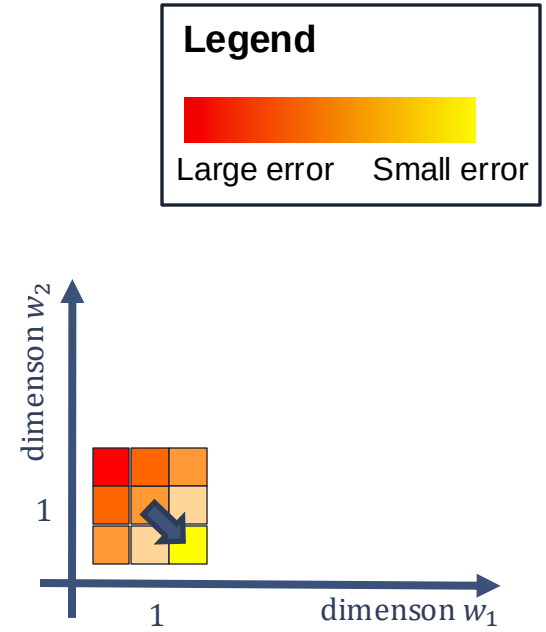
Gradient Optimisation Method: Multiple inputs

- We can compute the error for these nearby weights, as before.
- Given that we want to decrease the error, what would be a good update to the weights?



Gradient Optimisation Method: Multiple inputs

- From a given starting point, we want to get to a place which is as low as possible.
- A good strategy is to walk downhill in the steepest direction in the local area→Gradient Descent

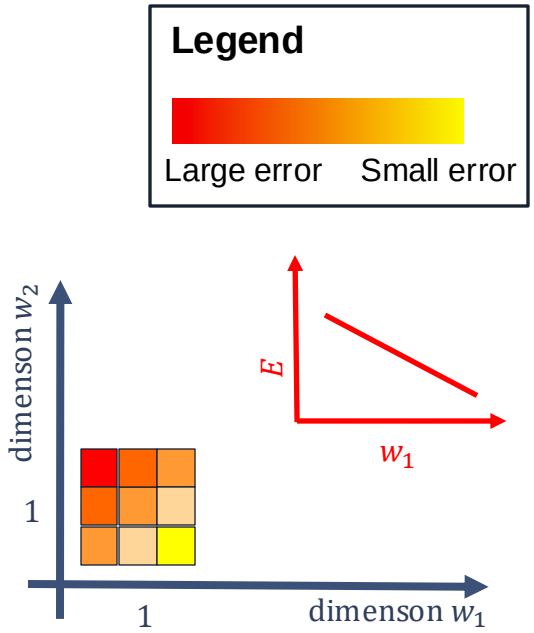


Gradient Optimisation Method: Multiple inputs

- In the local neighbourhood, we need to measure how the error changes as each weight changes ("error gradients")
- The change in error with respect to a small change in w_1 can be written:

$$\frac{\partial E}{\partial w_1}$$

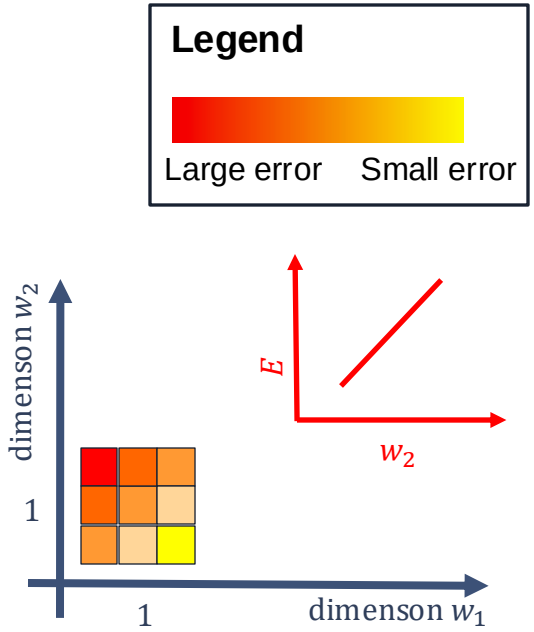
The partial derivative of E with respect to w_1



Gradient Optimisation Method: Multiple inputs

- Similarly, the change in error with respect to a small change in w_2 can be written:

$$\frac{\partial E}{\partial w_2}$$

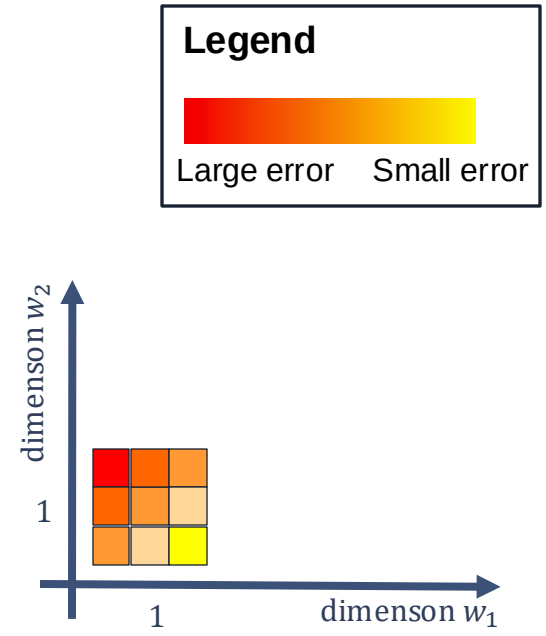


Gradient Optimisation Method: Multiple inputs

- In our example, we can see that as w_1 increases, E ***decreases***, and as w_2 increases, E ***increases***:

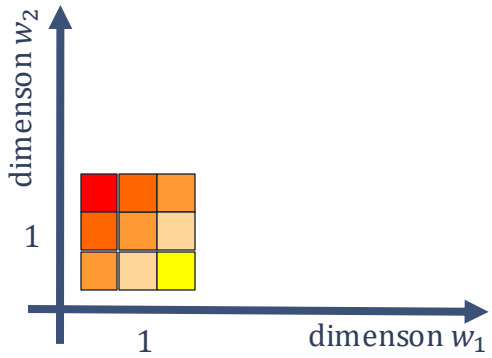
$$\frac{\partial E}{\partial w_1} < 0 \quad \textit{negative slope}$$

$$\frac{\partial E}{\partial w_2} > 0 \quad \textit{positive slope}$$



Gradient Optimisation Method: Multiple inputs

- By calculating the partial derivatives with respect to the weights, we obtain information about how the weights should be updated.
- This depends on the error/activation function surface being smooth (i.e. differentiable with respect to the weights).



Gradient Descent Method

- Gradient Descent is an optimisation algorithm that aims to minimise a function by iteratively moving in the direction of the steepest descent as defined by the negative of the gradient.

$$For\ all\ i, j:$$
$$w_{ij} = w_{ij} - \alpha \frac{\partial Error}{\partial w_{ij}}$$

learning rate

Gradient Descent Method

- Given a training dataset: $\{(x^1, y^1), \ldots, (x^n, y^n)\}$,
- And error function: $\mathrm{E} = \frac{1}{2}(y - o)^2$
- Using derivatives we can check how the error changes as the output of the ANN changes:

$$\frac{\partial E}{\partial o} = o - y$$

Gradient Descent Method

- OK, so we can calculate how the error changes as the output changes:

$$\frac{\partial E}{\partial o} = o - y$$

- But what we really care about is how the error changes as the weights change:

$$\frac{\partial E}{\partial w_i} = ?$$

Neural Network

Backpropagation

Gradient descent and Backpropagation

- Backpropagation, short for backward propagation of errors, is an algorithm for supervised learning of ANNs.
- Given an ANN and an error function, the method calculates the gradient of the error function with respect to the neural network's parameters.

Gradient descent and Backpropagation

- We can break $\frac{\partial E}{\partial w_i}$ and $\frac{\partial E}{\partial b_i}$ into its component parts
- The weights and the bias affect the input to a neuron, which affect the output of this neuron, which affect the error value.
- We can use the "chain rule" of calculus: $\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$
  - z is a variable that depends on y
  - y is a variable that depends on x
  - z depends on x indirectly, through the intermediate variable y.

Coming Next:

Backpropagation and Gradient Descent for Perceptron